We also never specified a type for the functional argument or the return value, which should have the same type, namely a function that takes arbitrary arguments and returns void.

Put it all together, and you get this:

chapter_13/04/app/javascript/controllers/search_controller.ts

```
  basicSubmit(): void {
    if (this.inputTarget.value === "") {
      this.reset()
    } else {
      this.formTarget.requestSubmit()
    }
  }

  submit = debounce(this.basicSubmit.bind(this))
}

type Debounceable = (...args: any[]) => void

function debounce<T extends Debounceable>(
  functionToDebounce: T,
  wait = 300
): T {
  let timeoutId: number | undefined = undefined

  return ((...args) => {
    clearTimeout(timeoutId)
    timeoutId = window.setTimeout(() => {
      timeoutId = undefined
      functionToDebounce(...args)
    }, wait)
  }) as T
}
```

It turns out to be a little easier to manage this typing if `debounce` is an external function rather than a method, which is fine. The `timeoutId` is now explicitly declared as a union type of a number and undefined, and it's set to `undefined` both of the places it's set.

I've declared a type called `Debounceable`, which is just a function that takes